Operating Systems

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict (for example: fast response and many users on one machine)
- Which requirements are critical to patient safety
- How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment

You are expected to **rank the requirements of each environment and justify the ranking**.

Alternative Concept and Complexity Analysis

If the whole hospital used a single general-purpose operating system on every device, instead of a different type of OS for each environment, what would the solution be? Evaluate the worst-case alarm response time (unbounded scheduling delay versus a guaranteed deadline), the memory needed on each device, and the administration effort.

Evidence Expected

A ranked requirement table for every environment with a justification for each ranking.

TASK 2 – Judge the Type of Operating System for Each Environment **3 Marks**

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

Discuss:

- Mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld systems
- Which type best fits the monitors, tablets, workstations, database server, imaging server and partner link
- Symmetric versus asymmetric multiprocessing for the imaging server
- A clustered system with fail-over for the database server

Alternative Concept and Complexity Analysis

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be? Evaluate the availability (a single point of failure versus fail-over within seconds), the cost, and how throughput grows with the number of processors (time) and the memory available (space).

Evidence Expected

A justified choice of operating-system type for each environment, with the rejected alternatives and reasons.

TASK 3 – Evaluate Operating-System Structures **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate the possible structures of the operating system.

Discuss:

- Simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures
- The effect of each structure on reliability, performance and ease of updating
- Which structure suits the patient monitors and which suits the database server
- How Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured

Alternative Concept and Complexity Analysis

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be? Evaluate the time cost of a service request (one system call versus message passing between servers), the memory taken by the kernel, and the effect of a faulty driver on the whole system.

Evidence Expected

A comparison of structures against the hospital's requirements, with a defended choice for two environments.

TASK 4 – Evaluate the Services an Operating System Must Provide **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate the services each environment needs.

Discuss:

- User interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security
- Which services can be removed from the bedside monitor to make it smaller and faster
- Which services are essential for the database server
- How the OS protects one patient's records from another user's programs

Alternative Concept and Complexity Analysis

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be? Evaluate the memory and storage used by the extra services (space), the start-up time and the battery life (time).

Evidence Expected

A service table for each environment with justified inclusions and exclusions.

TASK 5 – Examine System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Examine how a program obtains OS services.

Discuss:

- The categories of system calls (process control, file management, device management, information maintenance, communication, protection)
- What happens step by step when a program calls read(): library call, trap, mode switch, kernel work, return
- How parameters are passed to the kernel (registers, a block in memory, the stack)
- How strace shows the system calls made by a program

Alternative Concept and Complexity Analysis

If programs called the C library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be? Evaluate the number of system calls for a file of n bytes (about n / 4096 with library buffering versus n / b for a buffer of b bytes) and the extra buffer space used by the library.

Evidence Expected

A step-by-step explanation of a system call with strace evidence from a real program.

TASK 6 – Evaluate the Cost of System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the number of system calls affects performance.

Discuss:

- Why each system call costs much more than an ordinary function call
- How the buffer size changes the number of read() and write() calls when the database copies a file

- How library buffering (fread, fwrite) reduces the number of system calls
- The best buffer size for the hospital's backup program, using your measurements

Complete the Code

Complete TODO 1 below using the read() and write() system calls, and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
    //   stop when read returns 0 (end of file) or -1; write exactly the n bytes that
    //   were read

    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");     /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Expected output:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Run the program under strace -c and compare the number of read() calls reported with the program's output.

Alternative Concept and Complexity Analysis

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be? Evaluate the number of system calls (a few versus about n / b), the copying between kernel and user space, and the memory used for very large files.

Evidence Expected

Measured system-call counts and times for different buffer sizes, with a justified recommendation.

TASK 7 – Evaluate Process Control System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how a command shell on the database server starts and manages programs.

Discuss:

- What fork(), exec(), wait() and exit() each do
- How a shell runs a command, and how it obtains the command's exit status
- What a zombie and an orphan process are, and how the OS deals with them
- Why fork() followed by exec() is used instead of a single "create process" call

Complete the Code

Complete TODO 1 below using fork(), execlp() and waitpid(), and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout);                          /* empty the buffer before fork() */
    // TODO 1: Create a child with fork(). In the child, replace the program with
    //   execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
    //   fails. In the parent, wait for the child with waitpid() and print "job 1 exit
    //   status N" using WEXITSTATUS



    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);                  /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explain why stdout is flushed before each fork(). Then remove the second fflush(stdout), change _exit(3) to exit(3), redirect the output to a file and explain why "shell: running job 2" now appears twice.

Alternative Concept and Complexity Analysis

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be? Evaluate the time to start 1,000 commands, the memory copied or shared (copy-on-write), and the isolation between commands.

Evidence Expected

A working program that runs commands like a shell, with process diagrams and a reasoned evaluation.

TASK 8 – Evaluate CPU States and Protection — 3 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the hardware protects the hospital system.

Discuss:

- User mode and kernel mode, privileged instructions and the mode bit
- Interrupts, traps and the timer interrupt
- What would happen to the monitors if any program could disable interrupts
- How the hardware stops one program from reading another program's memory

Alternative Concept and Complexity Analysis

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be? Evaluate the worst-case delay before an abnormal reading is noticed (the polling interval versus the interrupt latency) and the CPU time and power wasted by polling.

Evidence Expected

A reasoned explanation of CPU modes, interrupts and protection, applied to the monitors.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how data moves between storage levels and devices.

Discuss:

- The levels of the memory hierarchy (registers, cache, main memory, SSD, hard disk, network storage) with typical access times and sizes
- Where MRI images, patient records and monitor readings should be kept
- Programmed I/O, interrupt-driven I/O and DMA, and which suits the MRI scanner
- The effect of caching on the speed of record look-up

Alternative Concept and Complexity Analysis

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be? Evaluate the number of interrupts and the CPU time for a 500 MB image, and the memory buffers needed in each method.

Evidence Expected

A justified data-placement plan and I/O method for each device, using typical access times.

TASK 10 – Measure and Compare **4 Marks**

Bloom's Level: K5 – Evaluate

Measure the behaviour of your programs on a Linux system.

Discuss:

- The number of system calls made by your copy program for different buffer sizes (strace -c)
- The time spent in user mode and in kernel mode (the time command)
- The time to create 1,000 processes with fork() and to create 1,000 threads
- How the measurements support or contradict your earlier judgements

Alternative Concept and Complexity Analysis

If threads were created instead of processes for the measurements, what would the result be? Evaluate the creation time for 1,000 threads versus 1,000 processes and the memory used by each, and explain the difference using what each one shares.

Evidence Expected

Measured results in tables, compared with expectations, with explained differences.

TASK 11 – Recommend and Defend the Operating-System Plan **4 Marks**

Bloom's Level: K5 – Evaluate

Recommend the complete operating-system plan for the hospital.

Discuss:

- The type and structure of operating system for each environment
- The services, protection and I/O methods each will use
- The limitations and risks of your plan
- Whether the plan remains suitable for the group of three hospitals

Your recommendation must be based on **evidence from your own analysis and measurements**.

Alternative Concept and Complexity Analysis

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be? Evaluate the effect on response time for the monitors, the memory and processor overhead of virtualisation, and the effect on reliability.

Evidence Expected

A technically justified plan supported by analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3

Task	Bloom's Level	Marks
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

TASK 1-Evaluate the Requirements of Each Computing Environment

Introduction :

A modern smart hospital is a complex computing environment that depends on different types of computer systems for providing efficient healthcare services. Unlike traditional hospitals, smart hospitals use advanced computing technologies such as patient monitoring systems, electronic health record systems, medical imaging systems, mobile devices and interconnected communication networks.

Within a hospital setting, different computing devices perform different tasks. For instance, bedside patient monitoring systems continuously collect patient information, nurses use handheld tablets to update medical records, doctors use desktop workstations for diagnosis, and servers manage large amounts of medical data.

Each computing environment has unique requirements. Some systems require extremely rapid response time, while others require high storage capacity, security, reliability or processing power. Thus, choosing a suitable operating system is an important decision.

An operating system acts as an interface between hardware and application software. It manages CPU resources, memory, storage devices, security mechanisms and communication between different components. In a smart hospital, choosing an inappropriate operating system may affect system performance and patient safety.

A single general-purpose operating system cannot satisfy all hospital requirements because different devices have different priorities. A patient monitoring device requires real-time response, whereas a database server requires continuous availability and high data processing capability. Therefore, every computing environment must be evaluated separately, and the operating system should be selected based on the specific requirements of that environment.

1. Evaluation of Different Computing Environments

1.1 Bedside Patient Monitoring System

A bedside patient monitoring system is one of the most critical computing environments in a smart hospital. These systems continuously monitor patient conditions and collect information from medical sensors.

The OS used in such systems must provide fast and predictable responses. When abnormal conditions are detected, the system must immediately generate an alarm. Even a small delay may create serious consequences.

Major Requirements:

1. Response Time

Response time is the most important requirement for patient monitoring systems. The OS must process sensor data and respond immediately without unnecessary delays.

Real-time operating systems are suitable because they provide deterministic scheduling and guarantee that critical tasks are completed within a specified time.

2. Reliability

Patient monitoring systems must operate continuously without failures. The OS should provide fault handling mechanisms and stable performance.

A system crash or unexpected shutdown may interrupt patient monitoring. Therefore, reliability is a major requirement.

3. Security

Medical information is highly sensitive. Unauthorized access or modification of patient data must be prevented.

The OS should provide authentication, access control and secure communication.

Recommended Operating System Approach:

A Real-Time Operating System (RTOS) is the most suitable choice for bedside monitoring systems because it provides:

- Fast response
- Predictable execution
- Efficient resource usage
- High reliability

1.2 Nurses' Handheld Tablets

Nurses use handheld tablets to access patient information, update medical records and communicate with doctors.

These devices are portable and operate throughout long hospital shifts. Therefore, their operating system requirements are different from patient monitoring systems.

Key Requirements:

1. Battery Efficiency

Since nurses carry tablets while moving between different departments, battery life is an important factor.
The OS should optimize power consumption by controlling background processes and hardware usage.
2. Security
Handheld devices contain confidential patient information. Strong security mechanisms are required to prevent unauthorized access.
Important security features include:

- User authentication
- Data encryption
- Secure application management

3. Mobility Support
The OS should support wireless communication technologies such as Wi-Fi and mobile networks.
It should allow nurses to access information from different locations inside the hospital.
Recommended Operating System Approach:
Mobile operating systems are suitable for handheld tablets because they provide:

- Power management
- Wireless connectivity
- Touch-based user interfaces
- Application support

1.3 Doctors' Desktop Workstations
Doctors' desktop workstations are used for diagnosis, patient record analysis, medical report preparation and communication with other healthcare professionals. These systems require a balance between performance, usability and security.
Unlike patient monitoring systems, desktop workstations do not require extremely fast real-time response. However, they must provide smooth operation because doctors may need to use multiple applications simultaneously.
Key Requirements:
1. Multitasking Capability
Doctors often use multiple applications at the same time, such as electronic health record software, medical image viewers and communication tools.

The OS should support efficient multitasking by managing CPU scheduling and memory allocation.

2. User Interface Support

A user-friendly graphical interface is necessary because doctors interact with many software applications during diagnosis and treatment.

The OS should provide:

- Easy navigation
- Fast application access
- Support for medical software

3. Security and Authentication

Doctors access confidential patient information. Therefore, the operating system must provide strong user authentication and access control.

Security mechanisms include:

1. Password protection
2. User permission management
3. Data encryption

Recommended Operating System Approach:

A general-purpose desktop operating system with strong security features is suitable for doctors' workstations because it provides:

1. Multitasking support
2. Application compatibility
3. User-friendly interface
4. Security management

1.4 Central Database Server

The central database server is a particularly important components of the smart hospital environment. It stores electronic health records, patient history, laboratory reports and other medical information.

The database server must operate continuously because doctors and nurses depend on accurate patient information.

Key Requirements:

1. High Availability

The server should remain available throughout the day because healthcare services operate continuously.

Downtime can prevent healthcare workers from accessing important patient information.

2. Data Security

Medical records contain private information. The OS must protect data from unauthorized access.
Security features include:

1. Access control
2. Encryption
3. User authentication
4. Regular security updates

3. High Throughput
A hospital database may receive requests from many users simultaneously.
The OS must effectively manage:

1. CPU resources
2. Memory usage
3. Network communication
4. Storage operations

Recommended Operating System Approach:
A server operating system is suitable for database servers because it provides:

1. High reliability
2. Large memory support
3. Network management
4. Security features
5. Fault tolerance

TASK 2- Judge the Type of Operating System for Each Environment

Introduction

A modern smart hospital consists of multiple computing environments that perform different tasks. Each environment has unique requirements related to performance, reliability, scalability, security and response time. An operating system manages hardware resources and provides services required by applications. However, different operating system types are designed for different purposes. For instance, a real-time operating system is designed for applications requiring immediate response, while a distributed operating system focuses on managing multiple connected computers.

Selecting an unsuitable operating system can reduce system efficiency and may create serious problems in healthcare environments. Therefore, the operating system type must be selected based on the workload and purpose of each hospital system.

The major operating system types considered for the smart hospital are:

1. Mainframe Operating System
2. Desktop Operating System
3. Multiprocessor Operating System
4. Distributed Operating System
5. Clustered Operating System
6. Real-Time Operating System
7. Handheld Operating System

1. Comparison of Different Operating System Types

Operating System Type	Description	Advantages	Limitations
Mainframe OS	Designed for large-scale data processing using powerful computers	High reliability, handles huge workloads	Expensive and requires specialized hardware
Desktop OS	Designed for personal computers and user interaction	Easy to use, supports many applications	Limited scalability for large workloads
Multiprocessor	Uses multiple CPUs	Increased	More complex

Operating System Type	Description	Advantages	Limitations
OS	in a single system	performance and parallel processing	hardware management
Distributed OS	Manages multiple independent computers connected through a network	Resource sharing and scalability	Network dependency
Clustered OS	Connects multiple computers to work together as one system	High availability and fault tolerance	Higher setup complexity
Real-Time OS	Provides predictable response within deadlines	Fast response and reliability	Limited general-purpose features
Handheld OS	Designed for portable devices	Low power usage and mobility support	Limited processing capability

2. Selection of Operating System Type for Hospital Environments

2.1 Patient Monitoring System

Recommended Operating System:

Hard Real-Time Operating System:-

A patient monitoring system continuously collects information from sensors and must respond immediately when abnormal conditions occur. A hard real-time system guarantees that critical operations are completed within a fixed deadline.

Reasons:

1. Immediate alarm generation
2. Predictable response time
3. High reliability
4. Continuous operation

Rejected Alternatives:

Desktop Operating System:

Not suitable because normal desktop systems cannot guarantee response deadlines.
Distributed Operating System:
Not preferred because network delays may affect emergency responses.

2.2 Nurses' Handheld Tablets
Recommended Operating System:
Handheld Operating System
Nurses require portable devices for accessing patient records and updating information.
Reasons:

1. Battery optimization
2. Wireless connectivity
3. Touch interface support
4. Mobile application support

Rejected Alternatives:
Mainframe OS:
Not suitable because handheld devices do not require large-scale processing.
Real-Time OS:
Not required because tablet operations are not safety-critical.
2.3 Doctors' Workstations
Recommended Operating System:
Desktop Operating System
Doctors use workstations for diagnosis, documentation and running medical applications.
Reasons:

1. User-friendly interface
2. Multitasking support
3. Application compatibility
4. Security management

Rejected Alternatives:
Handheld OS:
Limited performance compared with desktop systems.
Real-Time OS:
Not required because doctor applications do not always require deadline-based execution.
3. Detailed Evaluation of Operating System Selection

Choosing the correct operating system type for each hospital environment is important because every system has different objectives. A hospital cannot depend on a single operating system because the requirements of a medical monitoring device are completely different from those of a database server or a user workstation.

Importance of Real-Time Operating Systems in Healthcare

Real-time operating systems play an important role in healthcare applications where time accuracy is critical. These operating systems are designed to complete tasks within a predefined time limit. Unlike general-purpose operating systems, real-time systems focus on predictable execution rather than maximum user convenience.

In a smart hospital, hard real-time operating systems are suitable for patient monitoring equipment because missing a deadline can directly affect patient safety.

These systems provide:

1. Deterministic scheduling
2. Fast interrupt handling
3. Reliable task execution
4. Minimum response delay

Soft real-time operating systems can be used in applications where occasional delays are acceptable, such as medical data visualization systems and communication applications.

4. Role of Distributed Operating Systems in Smart Hospitals

A distributed operating system manages multiple computers connected through a network and makes them appear as a single integrated system.

In a smart hospital, distributed systems are useful because different departments generate and share large amounts of information.

For instance, laboratory systems, pharmacy systems, patient records and administrative systems can communicate through a distributed environment.

Benefits:

Resource Sharing:-

Different departments can share computing resources, storage systems and databases effectively.

Scalability:

Additional systems can be added when hospital requirements increase.

Improved Performance:

Tasks can be distributed among multiple computers, reducing the workload on individual systems.

Limitations:

1. Dependence on network connectivity
2. Security challenges
3. More complex system management

Therefore, distributed operating systems are suitable for hospital communication networks but are not ideal for emergency monitoring systems where immediate response is required.

TASK 3- Evaluate Operating System Services Required for Each Computing Environment

Introduction

An operating system provides various services that help application programs communicate with hardware resources and perform operations effectively. These services are responsible for managing processes, memory, files, devices, security and communication.

In a smart hospital environment, operating system services play an important role because different computing systems have distinct requirements. A patient monitoring system requires fast process management, a database server requires efficient file and storage management, and handheld devices require power management and security services.

The OS acts as the backbone of hospital computing infrastructure by ensuring reliable and secure operation of all systems.

The major operating system services required in a smart hospital include:

1. Program Execution
2. Process Management
3. Memory Management
4. File System Management
5. Device Management
6. Input/Output Management
7. Security and Protection
8. Communication Services
9. Error Detection and Handling
10. Resource Allocation

1. Evaluation of Operating System Services

1.1 Program Execution Service

Program execution is one of the basic services provided by an operating system. It loads programs into memory and manages their execution. Within a hospital setting, many applications such as patient monitoring software, electronic health record systems and medical imaging applications need reliable execution.

Importance:

1. Allows medical applications to run effectively
2. Provides proper resource allocation

3. Prevents application conflicts

Example:

When a doctor opens a patient record application, the operating system loads the required files into memory and provides CPU resources for execution.

1.2 Process Management Service

Process management controls the execution of multiple processes running in the system.

A hospital environment contains multiple simultaneous operations. For instance, a database server may handle hundreds of user requests while monitoring systems continuously collect patient data.

Functions of Process Management:

1. Process creation
2. Process scheduling
3. Process synchronization
4. Process termination

Application in Hospital:

Patient Monitoring System:

Process management ensures that monitoring tasks receive priority over less important background operations.

Database Server:

It manages multiple user requests effectively.

1.3 Memory Management Service

Memory management controls the allocation and usage of main memory. Different hospital systems have different memory requirements. Medical imaging servers require large memory capacity, while handheld devices require efficient memory utilization.

Functions:

1. Memory allocation
2. Memory deallocation
3. Virtual memory management
4. Protection between processes

Application:

MRI imaging servers handle large image files. Efficient memory management prevents system slowdown and improves processing speed.

1.4 File System Management Service

File system management organizes and controls data storage.

In hospitals, huge amounts of information such as patient records, medical reports and imaging files are stored every day.
Requirements:

1. Secure storage
2. Fast data retrieval
3. Backup support
4. Access control

Application:
The database server uses file management services to store and retrieve electronic health records effectively.

1.5 Device Management Service
Device management allows the operating system to control hardware devices connected to the system.
A modern smart hospital contains many devices including:

1. Patient sensors
2. Medical scanners
3. Printers
4. Storage devices
5. Network equipment

The OS provides device drivers that allow applications to communicate with hardware.
2. Advanced Operating System Services in a Smart Hospital Environment
Apart from basic operating system services, a smart hospital requires advanced services to maintain efficiency, reliability and security. These services help different healthcare systems communicate properly and ensure continuous operation.
2.1 Security and Protection Services
Security and protection are among the most important operating system services in a healthcare environment because hospitals store highly confidential information.
Patient records include personal details, medical history, laboratory reports and treatment information. Unauthorized access or modification of this data can create serious problems.
The OS provides security mechanisms to protect hospital resources.
Important Security Functions:
User Authentication

Authentication verifies the identity of users before allowing access to hospital systems.

Examples:

1. Doctors use login credentials to access patient records.
2. Nurses use authorized accounts to update medical information.

Access Control

Access control determines what actions a user can perform.

For example:

1. Doctors may view and update treatment details.
2. Administrative staff may only manage appointment information.

Data Encryption

Encryption converts data into a protected format so that unauthorized users cannot understand it.

This is important when patient information is transferred between departments or partner hospitals.

Application in Smart Hospital:

Security services are required in:

1. Database servers
2. Doctors' workstations
3. Mobile healthcare devices
4. Communication systems

2.2 Communication Services

Communication services allow different systems to exchange information effectively.

A modern smart hospital contains many interconnected devices and applications. Communication between these systems is necessary for providing fast healthcare services.

Examples:

Patient Monitoring Communication

Patient monitoring devices continuously send sensor information to healthcare systems.

The OS manages communication between hardware devices and monitoring applications.

Hospital Network Communication:

Different departments such as laboratories, pharmacies and emergency units share information through networks.

The OS manages:

1. Network connections

2. Data transmission
3. Resource sharing

Partner Hospital Communication

When hospitals exchange patient information, communication services ensure secure and reliable data transfer.

1. Relationship Between Operating System Services and Hospital Requirements

Hospital Requirement	Required OS Service	Purpose
Patient safety	Process management + Real-time scheduling	Provides quick response
Patient record protection	Security service	Prevents unauthorized access
Medical image processing	Memory and file management	Handles large data files
Device operation	Device management	Controls medical equipment
Data exchange	Communication service	Connects departments
System reliability	Error detection and recovery	Prevents service interruption

TASK 4-**Analyze Process Management Requirements in Different Computing Environments**

Introduction

Process management is a particularly important functions of an operating system. A process is a program that is currently executing and requires resources such as CPU time, memory and input/output devices.

In a smart hospital environment, thousands of processes execute simultaneously. Patient monitoring applications collect real-time sensor data, doctors access electronic health records, medical imaging systems process large files, and database servers handle multiple user requests. The OS is responsible for managing these processes effectively. It decides which process receives CPU resources, maintains communication between processes and prevents conflicts between different applications.

Effective process management ensures:

1. Fast response time
2. Efficient resource utilization
3. System reliability
4. Better performance
5. Patient safety

1. Process Management Requirements in Hospital Computing Environments

1.1 Patient Monitoring System

Patient monitoring systems are safety-critical systems that continuously collect information from medical sensors.

The OS must prioritize monitoring processes because they require immediate execution.

Process Management Requirements:

1. Priority Scheduling

Patient monitoring processes should receive higher priority compared with normal background applications.

For example:

A heart-rate monitoring process must execute before a less important data backup process.

2. Real-Time Scheduling

Real-time scheduling ensures that critical tasks are completed within a fixed time limit.

Benefits:

1. Predictable response

2. Minimum delay
3. Improved patient safety

3. Process Synchronization

Multiple sensors may provide data simultaneously. The OS must synchronize these processes to avoid incorrect information.

1.2 Nurses' Handheld Tablets

Handheld tablets execute multiple applications such as patient record systems, communication applications and authentication services.

Process Management Requirements:

1. Efficient Multitasking

The OS must manage multiple applications while maintaining smooth performance.

Example:

A nurse may update patient records while receiving alerts from the hospital system.

2. Power-Aware Process Management

Since tablets depend on batteries, unnecessary background processes should be controlled.

Benefits:

1. Increased battery life
2. Reduced processor usage
3. Better device performance

3. Secure Process Isolation

Medical applications should run separately to prevent one application from affecting another.

1.3 Doctors' Desktop Workstations

Doctors' workstations require efficient process management because multiple medical applications may run simultaneously.

Examples:

1. Patient record software
2. Medical image viewers
3. Communication applications
4. Report generation tools

Process Management Requirements:

1. Multilevel Scheduling

The OS should assign different priorities to different applications.

For example:

A medical diagnosis application receives higher priority than a document editing application.

2. Resource Sharing

Processes must share CPU, memory and storage resources effectively.

3. Process Protection

One faulty application should not affect other running applications.

1.4 Database Server Process Management

The database server handles thousands of requests from doctors, nurses and hospital systems.

Process Management Requirements:

1. Concurrent Process Execution

The server must handle multiple user requests at the same time.

Example:

Many doctors accessing patient records simultaneously.

2. Deadlock Management

Multiple processes may compete for the same resources. The OS must prevent deadlock situations.

Deadlock occurs when processes wait indefinitely for resources held by each other.

3. Load Management

The OS must distribute workload effectively to maintain server performance.

1. Comparison of Process Management Requirements:

Environment	**Main Process Requirement**	**Suitable Scheduling Approach**
Patient Monitoring	Immediate response	Real-Time Scheduling
Handheld Tablets	Power-efficient multitasking	Mobile Process Management
Doctors' Workstation	Multiple applications	Multilevel Scheduling
Database Server	Concurrent processing	Time-Sharing Scheduling
MRI Server	Parallel execution	Multiprocessor Scheduling

TASK 5: Analyze Memory Management Requirements in Different Computing Environments

Introduction

Memory management is a particularly important responsibilities of an operating system. It controls how main memory is allocated, used and released by different processes. Efficient memory management ensures that applications receive sufficient memory resources while preventing conflicts between processes.

A modern smart hospital environment contains different computing systems that require different memory management approaches. Patient monitoring systems require fast memory access for real-time operations, database servers require large memory capacity for storing and processing medical records, and imaging servers require advanced memory management techniques for handling large medical files.

The OS performs memory management through various techniques such as:

1. Memory allocation
2. Memory protection
3. Virtual memory management
4. Paging
5. Segmentation
6. Cache management

Proper memory management improves system performance, reliability and security.

1. Memory Management Requirements in Hospital Computing Environments

1.1 Patient Monitoring System

Patient monitoring systems continuously collect data from medical sensors and process information in real time.

The memory management system must provide quick access to critical data because delays can affect patient safety.

Memory Requirements:

1. Fast Memory Access

Real-time applications require rapid access to sensor data. The OS must ensure that important processes remain available in memory.

2. Memory Protection

Patient monitoring applications must be protected from interference by other applications.

Memory protection prevents:

1. Unauthorized access
2. Data corruption
3. Application crashes

3. Efficient Memory Allocation

The OS should allocate memory dynamically based on the requirement of monitoring applications.

Suitable Memory Technique:

Real-time memory allocation techniques are suitable because they provide predictable performance.

1.2 Nurses' Handheld Tablets

Handheld tablets have limited memory resources compared with servers. Therefore, efficient memory utilization is important.

Memory Requirements:

1. Low Memory Consumption

The OS should minimize unnecessary memory usage to improve battery life.

2. Multitasking Support

Nurses may use multiple applications simultaneously:

1. Patient records
2. Communication applications
3. Medical alerts

The OS must manage memory allocation between these applications.

3. Application Isolation

Each application should operate in its own memory space to prevent security issues.

Suitable Memory Techniques:

1. Paging
2. Virtual memory
3. Memory compression

1.3 Doctors' Desktop Workstations

Doctors' workstations run multiple medical applications requiring efficient memory management.

Applications include:

1. Electronic health records
2. Medical image viewers
3. Report generation software

Memory Requirements:

1. Multitasking Memory Management

The OS should effectively switch between different applications.

2. Virtual Memory Support

Virtual memory allows the system to run large applications even when physical memory is limited.

3. Memory Protection

Patient information stored in memory must remain secure.

1.4 Database Server

The database server stores and processes large amounts of healthcare information.

Memory management is critical because database performance depends heavily on efficient memory usage.

Memory Requirements:

1. Large Memory Capacity

The server requires sufficient memory to handle:

1. Patient records
2. Transaction processing
3. Multiple user requests

2. Cache Management

Frequently accessed data can be stored in cache memory to improve response time.

3. Memory Sharing

Multiple database processes need controlled access to shared memory.

Suitable Memory Techniques:

1. Large memory allocation
2. Cache optimization
3. Virtual memory management

1.5 MRI Imaging Server

MRI imaging systems generate extremely large files that require significant memory resources.

Memory Requirements:

1. High-Speed Memory Processing

Large images must be processed quickly for diagnosis.

2. Efficient Data Handling

The OS must manage movement of image data between:

1. Main memory
2. Storage devices
3. Processing units

Task 5 – System-call demonstration (supporting code)

Save as task5_syscall_demo.c. This program calls open(), read(), write(), and close() to display a file.

```
#define _POSIX_C_SOURCE 200809L
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <file>\n", argv[0]);
        return 1;
    }

    int fd = open(argv[1], O_RDONLY);
    if (fd == -1) {
        perror("open");
        return 1;
    }

    char buffer[1024];
    ssize_t n;
    while ((n = read(fd, buffer, sizeof buffer)) > 0) {
        ssize_t written = 0;
        while (written < n) {
            ssize_t w = write(STDOUT_FILENO, buffer + written,
                              (size_t)(n - written));
            if (w == -1) {
                perror("write");
                close(fd);
                return 1;
```

```
            }
            written += w;
        }
    }

    if (n == -1) perror("read");
    if (close(fd) == -1) perror("close");
    return (n == -1) ? 1 : 0;
}
```

Compile and run:

```
gcc -Wall -Wextra task5_syscall_demo.c -o task5_demo
./task5_demo sample.txt
strace -e openat,read,write,close ./task5_demo sample.txt
```

Evaluation: open() obtains a file descriptor; read() transfers bytes into user space; write() sends them to standard output; close() releases the descriptor. strace provides observable evidence of the kernel boundary.

TASK 6 - Analyze File System Management Requirements in Different Computing Environments

Introduction

File system management is one of the major functions provided by an operating system. A file system controls how data is stored, organized, accessed and protected in storage devices.

In a smart hospital environment, large amounts of digital information are generated every day. These include electronic health records, medical images, laboratory reports, prescriptions, sensor data and administrative documents.

The OS manages these files by providing:

1. File organization
2. Storage allocation
3. Access control
4. Data protection
5. Backup and recovery
6. File retrieval mechanisms

Different hospital computing environments have different file system requirements. A patient monitoring system requires fast storage access, a database server requires reliable data management, and an imaging server requires support for extremely large files.

Thus, choosing an appropriate file system management approach is essential for maintaining hospital efficiency and reliability.

1. File System Requirements in Different Hospital Environments

1.1 Patient Monitoring System

Patient monitoring systems continuously collect data from medical sensors. This data includes patient vital information, alerts and monitoring records.

The file system must handle continuous data storage while maintaining fast access.

File System Requirements:

1. Fast Data Storage

Sensor information must be stored quickly without delays.

The OS should provide efficient storage operations so that monitoring data can be recorded continuously.

2. Data Reliability

Patient monitoring records must not be lost because they are important for diagnosis and treatment decisions.

The file system should provide:

1. Error detection
2. Data consistency
3. Recovery mechanisms

3. Secure Data Access

Only authorized medical staff should access patient information.

Security mechanisms include:

1. User permissions
2. File access control
3. Encryption

Suitable File System Approach:

A reliable file system with strong security and fast access is suitable for patient monitoring systems.

1.2 Nurses' Handheld Tablets

Nurses use handheld devices to access patient records and update information.

Since these devices have limited storage capacity, efficient file management is required.

File System Requirements:

1. Storage Optimization

The OS should manage limited storage effectively by removing unnecessary temporary files and optimizing storage usage.

2. Mobile Data Synchronization

Information collected by nurses must synchronize with the central hospital database.

The file system should support:

1. Data transfer
2. File synchronization
3. Offline storage

3. Security Protection

If a tablet is lost or accessed by unauthorized users, patient information should remain protected.

Security features include:

1. File encryption
2. Device authentication
3. Secure storage

1.3 Doctors' Desktop Workstations

Doctors' workstations store and access various medical documents and applications.

Examples:

1. Patient reports
2. Prescription records
3. Diagnostic information
4. Medical images

File System Requirements:

1. Fast File Access

Doctors require quick access to patient records during diagnosis.

2. File Organization

The OS should organize files effectively so information can be retrieved easily.

3. Access Control

Different users should have different levels of file access.

Example:

1. Doctors can modify treatment records.
2. Administrative staff may only view appointment details.

1.4 Database Server

The database server manages a large collection of structured healthcare data.

File system performance directly affects database performance.

File System Requirements:

1. High Reliability

Database files must remain consistent even during failures.

The OS should support:

1. Journaling
2. Backup systems
3. Recovery mechanisms

2. Large Storage Support

The server must handle huge amounts of patient data generated over many years.
3. Efficient File Allocation
The file system should allocate storage effectively to reduce fragmentation and improve performance.
1.5 MRI Imaging Server
Medical imaging systems generate extremely large files compared with normal hospital data.
MRI scans require efficient storage and retrieval.

File System Requirements:
1. Large File Support
The file system must support storing large medical images without performance reduction.
2. High-Speed Access
Doctors require quick loading of images for diagnosis.
3. Storage Scalability
The system should allow additional storage capacity as the number of medical images increases.

Task 6 – Cost of system calls (completed TODO)

Save as task6_copy.c. The loop handles partial writes and counts every read call, including the final EOF read, matching the assignment's expected counts.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    if (bufsize == 0 || bufsize > sizeof buf) {
        errno = EINVAL;
        return -1;
    }

    int in = open(src, O_RDONLY);
```

```
    if (in < 0) return -1;

    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }

    long calls = 0;
    ssize_t n;

    while (1) {
        n = read(in, buf, bufsize);
        calls++;
        if (n == 0) break;
        if (n < 0) {
            if (errno == EINTR) continue;
            close(in); close(out);
            return -1;
        }

        ssize_t offset = 0;
        while (offset < n) {
            ssize_t w = write(out, buf + offset, (size_t)(n - offset));
            if (w < 0) {
                if (errno == EINTR) continue;
                close(in); close(out);
                return -1;
            }
            offset += w;
        }
    }

    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");
    if (!f) { perror("fopen"); return 1; }
    for (int i = 0; i < 10000; i++)
        fputc('A' + i % 26, f);
    fclose(f);
```

```
    size_t sizes[] = {1, 512, 4096};
    for (size_t i = 0; i < sizeof sizes / sizeof sizes[0]; i++) {
        long calls = copy_file("records.dat", "copy.dat", sizes[i]);
        if (calls < 0) { perror("copy_file"); return 1; }
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i], calls);
    }

    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Compile/run/trace:

```
gcc -Wall -Wextra task6_copy.c -o task6_copy
./task6_copy
strace -c -e read,write,openat,close ./task6_copy
```

Expected program output:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Note: strace counts may include runtime/library activity and may vary slightly by environment. Explain the measured results. Big-O: copying N bytes is O(N) time; buffer space is O(B). mmap alternative: map file pages and copy to destination; fewer explicit read/write calls, but page faults and mapping calls remain, and mapping very large files needs careful windowing/resources.

TASK 7 – EVALUATE PROCESS CONTROL SYSTEM CALLS

7.1 Introduction

A process is a program that is currently being executed by the operating system. Within a hospital setting, many processes run simultaneously to support important activities such as:

- Managing patient registration.
- Accessing electronic health records.
- Processing laboratory reports.
- Monitoring patients through medical devices.
- Managing MRI and CT scan operations.
- Running hospital billing and pharmacy applications.

The OS must manage these programs effectively and ensure that each process receives the necessary CPU time and system resources.

A command shell is a program that accepts commands from the user and requests the operating system to execute them. Examples include Bash, Zsh and other Linux shells.

When a user enters a command, the shell creates a child process, executes the required program and waits for the program to finish. It then collects the exit status to determine whether the command was successful.

7.2 Functions Used for Process Management

Four important system calls or library functions are fork(), exec(), wait() and exit().

1. fork()

The fork() system call creates a new child process from the calling parent process.

The child process initially receives a copy of the parent's process state.

Modern Linux systems use Copy-on-Write (COW), so physical memory pages are not immediately copied.

Return values:

- Returns a negative value if process creation fails.
- Returns 0 in the child process.
- Returns the child's process ID (PID) in the parent process.

2. exec()

The exec() family of functions replaces the current process program with another program.

The process ID remains the same, but the program's code, data and stack are replaced.

Common functions include:

- execl()
- execlp()
- execv()
- execvp()

3. wait()

- The wait() system call allows a parent process to wait until one of its child processes terminates.
- It collects the child's termination information and prevents the child from remaining as an unreaped zombie process.

The parent can examine the returned status using macros such as:

- WIFEXITED(status)
- WEXITSTATUS(status)
- WIFSIGNALED(status)
- WTERMSIG(status)

4. exit()

The exit() function terminates a process normally.

It performs standard C library cleanup, including flushing buffered standard I/O streams and executing registered termination handlers.

The value 0 normally indicates successful execution, whereas a non-zero value indicates an error or unsuccessful result.

The _exit() system call terminates the process immediately without performing normal C library stream flushing or registered exit-handler processing.

7.3 How a Shell Executes a Command

When a user enters a command, the shell performs the following operations:

4. Reads the command entered by the user.
5. Separates the command name and arguments.
6. Creates a child process using fork().
7. The child calls an exec() function to execute the required program.
8. The parent calls waitpid() to wait for the child.
9. The child terminates after completing the command.
10. The parent collects the exit status.
11. The shell displays the result or returns to the command prompt.

How the shell obtains the exit status

The parent process uses waitpid() to collect the termination status of the child.

For example:

```
int status;
waitpid(pid, &status, 0);
if (WIFEXITED(status))
{
   printf("Exit status: %d\n", WEXITSTATUS(status));
}
```

If the child exits with exit(3), the parent receives an exit status of 3.
The shell uses this status to determine whether the command completed successfully.

7.4 Zombie and Orphan Processes

Zombie process

A zombie is a process that has finished execution but whose parent has not yet collected its termination status.

The process no longer executes, but its process-table entry remains until the parent calls wait() or waitpid().

OS handling: Linux retains the child's exit information until the parent collects it. If the parent terminates without reaping the child, the orphaned zombie is eventually adopted by an appropriate system reaper, which can collect its status.

Orphan process

An orphan process is a process whose parent terminates before the child. Linux reparents the child to an appropriate subreaper or the system's init process (normally PID 1 in a traditional system).
The adopted process can continue executing and will eventually be reaped when it terminates.

Difference between zombie and orphan:

Feature	**Zombie**	**Orphan**
Execution	Already terminated	Still executing
Parent	Has not collected exit status	Original parent has terminated
Process table	Entry remains	Normal process entry remains
OS handling	Reaped using wait mechanism	Adopted by init or subreaper
Main issue	Wasted process-table entry	Parent-child relationship changes

In hospital servers, uncontrolled zombie processes may consume process-table resources. Therefore, server applications must correctly manage child processes.

7.5 Complete C Program Using fork(), execlp() and waitpid()

The following program demonstrates how a shell executes two commands.

Explanation:

5. Job 1 runs expr 6 * 7.
6. The command calculates 42 and prints it.
7. The command terminates successfully with exit status 0.
8. Job 2 terminates using _exit(3).
9. The parent collects exit status 3.

7.6 Alternative Concept: posix_spawn() and Threads

Instead of using fork() followed by exec(), Linux programs can use posix_spawn() to create and execute a new program.

Process vs thread execution

Feature	**fork() + exec()**	**posix_spawn()**	**Threads**
Execution unit	Separate process	Separate process	Thread in same process
Memory	Separate virtual address space	Separate virtual address space	Shared process memory
Isolation	High	High	Lower
Creation overhead	Relatively higher	Can be lower	Usually lower
Failure isolation	Better	Better	Shared-process failures possible
Best use	Independent commands	External program execution	Parallel work inside one application

Threads are not a direct replacement for independent shell commands because a thread does not automatically create a separate program image. Executing exec() in a multithreaded process replaces the entire process image, affecting all threads.

7.7 Conclusion

Process-control system calls are essential for managing multiple programs in a hospital server. fork() creates a child process, exec() loads a new program, waitpid() collects the child's exit status and exit() terminates a process.

Proper flushing of stdout prevents duplicate output. Zombie and orphan processes must also be managed correctly.

For the hospital environment, separate processes should be used for independent and security-sensitive programs. posix_spawn() may be selected for efficient external program launching, while threads are useful for internal parallel processing

Task 7 – Process control (completed TODO)

Save as task7_process.c. Includes error checks and both child jobs.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid;
    int status;

    printf("shell: running job 1\n");
    fflush(stdout);

    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        perror("execlp");
        _exit(127);
    }

    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid");
        return 1;
```

```
    }
    if (WIFEXITED(status))
        printf("job 1 exit status %d\n", WEXITSTATUS(status));
    else
        printf("job 1 did not exit normally\n");

    printf("shell: running job 2\n");
    fflush(stdout);

    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) _exit(3);

    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid");
        return 1;
    }
    if (WIFEXITED(status))
        printf("job 2 exit status %d\n", WEXITSTATUS(status));

    return 0;
}
```

Compile/run:

```
gcc -Wall -Wextra task7_process.c -o task7_process
./task7_process
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Why flush before fork? Buffered stdout content is copied into the child's memory at fork; flushing first prevents both processes from flushing the same buffered text. exit() flushes stdio; _exit() terminates without flushing inherited stdio buffers.

TASK 8 – EVALUATE CPU STATES AND PROTECTION

8.1 Introduction

A hospital information system manages highly important operations, including patient monitoring, medical imaging, electronic health records and emergency alerts.
The CPU must execute these applications without allowing one program to damage another program or interfere with the operating system.
Hardware protection mechanisms provide a secure environment by dividing CPU execution into different privilege levels.
The two important CPU execution modes are:

5. User mode.
6. Kernel mode.

The OS uses these modes to protect hardware, memory and system resources.

8.2 User Mode and Kernel Mode

User mode
User mode is a restricted execution mode in which normal application programs execute.
Examples include:

6. Hospital billing applications.
7. Patient registration software.
8. Web browsers.
9. Electronic medical record applications.
10. Image-viewing software.

Programs running in user mode cannot directly execute privileged instructions or access protected kernel memory.
If a user-mode program requires a protected service, it must request the service from the operating system through a system call.
Kernel mode
Kernel mode is the privileged execution mode of the CPU.
The OS kernel executes in this mode and can access protected hardware and system resources.
Kernel-mode operations include:

4. Managing memory.
5. Scheduling processes.
6. Controlling devices.
7. Handling interrupts.
8. Managing file systems.
9. Controlling CPU protection mechanisms.

Difference between the two modes

Feature	**User mode**	**Kernel mode**
Privilege	Restricted	High
Hardware access	Indirect	Direct where permitted
Privileged instructions	Not permitted	Permitted
Memory access	User-space memory	Kernel and permitted memory
Failure impact	Usually affects one process	Can affect the entire system
Example	Patient application	Linux kernel

8.3 Interrupts, Traps and Timer Interrupts

Interrupts

An interrupt is a signal that causes the CPU to temporarily stop its current execution and handle an important event.

Interrupts are generally asynchronous because they may occur independently of the currently executing instruction.

Examples:

10. A medical sensor sends a new reading.
11. A network interface receives data.
12. A storage device completes an operation.
13. A keyboard generates an input event.

In a hospital, interrupts allow the system to react quickly to new monitoring information.

Traps

A trap is a synchronous event generated by the execution of an instruction or by an exceptional condition.

Examples:

14. System calls.
15. Division-by-zero exceptions.
16. Invalid memory access.
17. Breakpoint instructions.

The processor transfers control to the appropriate kernel handler.

Comparison

Feature	Interrupt	Trap
Nature	Usually asynchronous	Synchronous
Source	Hardware/device event	Instruction or exception
Example	Network packet received	System call
Purpose	Handle external events	Handle software requests or exceptions

8.4 What Happens if Programs Can Disable Interrupts?

Suppose a hospital monitoring application is allowed to disable interrupts. It may prevent the CPU from responding to other important events.
Possible consequences include:

18. Medical sensor readings may not be processed on time.
19. Emergency alarms may be delayed.
20. The CPU scheduler may not regain control.
21. Network messages may remain unprocessed.
22. Other important applications may become unresponsive.

For instance, if a patient monitor detects an abnormal heart rate, the monitoring system must process the reading and generate an alert.
If another program disables interrupts for a long time, the monitoring event may not be handled promptly.
Therefore, disabling interrupts must be restricted to trusted kernel code and only for carefully controlled periods.
Evaluation: Allowing ordinary applications to disable interrupts would create a serious reliability and safety risk. Hardware privilege mechanisms protect the hospital system from this situation.

8.5 Alternative Concept: Polling Instead of Interrupts

Polling is a method in which the CPU repeatedly checks a device to determine whether new data is available.

For instance, a hospital monitoring application may check a sensor every 10 milliseconds.

Interrupt vs polling

Feature	**Interrupt-driven I/O**	**Polling**
CPU activity	Runs when event occurs	Repeatedly checks device
Response	Usually low latency	Depends on polling interval
CPU usage	Generally efficient for sparse events	Can waste CPU cycles
Power consumption	Usually lower	May be higher
Implementation	Requires interrupt support	Conceptually simpler
Suitability	Real-time monitoring	Periodic/simple device checks

8.6 Conclusion

CPU protection mechanisms are essential for hospital reliability and security. User mode restricts application access, while kernel mode allows trusted operating-system operations.

Interrupts and timer interrupts enable responsive event handling and fair CPU scheduling. The MMU and page tables prevent unauthorized memory access.

For hospital monitoring, interrupt-driven processing is preferable to continuous polling because it reduces CPU waste and can provide faster event detection. Critical monitoring applications should also use suitable real-time scheduling and carefully controlled kernel drivers

TASK 9 – EVALUATE THE MEMORY HIERARCHY AND I/O SYSTEMS

9.1 Introduction

A hospital information system handles a large amount of data every day. This includes MRI images, patient medical records, laboratory reports, prescriptions and continuous readings from patient-monitoring devices. The OS must manage data effectively by storing frequently used information in faster memory and less frequently used information in larger, slower storage devices.

The memory hierarchy is designed to balance three important factors:

23. Speed of data access.
24. Storage capacity.
25. Cost of storage.

The OS also uses different I/O techniques to transfer data between medical devices and computer memory.

The three main I/O techniques are:

26. Programmed I/O.
27. Interrupt-driven I/O.

9.2 Levels of the Memory Hierarchy

The memory hierarchy consists of several levels arranged according to speed, capacity and cost.

1. CPU Registers

Registers are small, high-speed storage locations inside the CPU.

They temporarily hold:

28. Instructions.
29. Memory addresses.
30. Arithmetic results.
31. Data currently being processed.

Typical access time is approximately 0.2–1 nanoseconds, depending on processor design.

Registers have very small capacity, but they provide the fastest access.

2. Cache Memory

Cache memory stores frequently used instructions and data so that the CPU does not need to access RAM for every operation.

There are three common levels:

32. L1 cache: Very fast and small.
33. L2 cache: Larger but slightly slower.
34. L3 cache: Larger, often shared between CPU cores.

3. Main Memory (RAM)

RAM stores currently executing programs and active data.

For example:

35. Hospital management applications.
36. Patient records currently being processed.
37. MRI images being viewed.
38. Temporary sensor readings.
39. Database cache pages.

4. Solid-State Drive (SSD)

An SSD stores data permanently using flash memory.

It is faster than a traditional hard disk and is suitable for databases, operating systems and frequently accessed medical images.

Typical SSD access latency ranges from approximately 50 microseconds to 1 millisecond, depending on the device and workload.

5. Hard Disk Drive (HDD)

A hard disk stores data on rotating magnetic disks.

It provides large storage capacity at a lower cost per byte than many SSDs.

Typical random access time is approximately 5–15 milliseconds.

It is useful for long-term storage and backup, although SSDs are generally better for databases requiring frequent random access.

Access time depends on network speed, server load, distance and storage technology.

It may range from a few milliseconds within a local data centre to much longer for remote cloud storage.

9.3 Where Should Hospital Data Be Stored?

Data placement plan

Data	Primary location	Secondary location	Reason
MRI images	SSD	HDD/NAS	Large files and frequent viewing
Patient records	SSD database	Replicated storage	Fast and reliable access
Monitor readings	RAM buffer	SSD/database	Immediate processing
Old medical images	HDD	Backup storage	Cost-effective archiving
Frequently accessed records	RAM cache	SSD	Faster look-up

9.4 Types of I/O Systems

Input/output systems allow computers to communicate with external devices.

1. Programmed I/O

In programmed I/O, the CPU continuously checks the status of a device and performs data transfers through programmed instructions.

The CPU remains involved in the operation.

Benefits:

40. Simple to implement.
41. Suitable for small data transfers.
42. Does not require a sophisticated DMA controller.

Limitations:

43. Wastes CPU time while waiting.
44. Inefficient for large data transfers.
45. Can reduce performance when multiple devices are active.

Example: A simple device that transfers a small amount of configuration data.

2. Interrupt-Driven I/O

In interrupt-driven I/O, the device sends an interrupt when it needs attention or when an operation is completed.
The CPU can perform other work while the device operates.
When an interrupt occurs, the CPU executes the appropriate interrupt handler.

Benefits:

46. Reduces continuous CPU polling.
47. Suitable for event-driven devices.
48. Supports responsive monitoring.

Limitations:

49. Too many interrupts may create CPU overhead.
50. Interrupt handling requires scheduling and synchronization.
51. Not ideal for very large transfers if every small unit generates an interrupt.

Example: A patient-monitoring device notifying the system when a new reading becomes available.

3. Direct Memory Access (DMA)

DMA allows a hardware controller to transfer data directly between a device and RAM without requiring the CPU to handle every individual data unit.
The CPU initially configures the transfer.
The DMA controller then performs the data movement.
When the transfer completes, an interrupt can notify the CPU.
Benefits:

52. High-speed data transfer.
53. Low CPU involvement.
54. Efficient for large blocks of data.
55. Allows the CPU to perform other tasks simultaneously.

Limitations:

56. Requires DMA-capable hardware and driver support.
57. Requires buffer management.
58. Incorrect DMA configuration can create memory and security problems.

9.5 Which I/O Method Is Best for an MRI Scanner?

An MRI scanner generates large amounts of image data.
For instance, a single examination may produce hundreds of megabytes of image information.
Using programmed I/O would require the CPU to continuously manage the transfer.
Interrupt-driven I/O could also create excessive overhead if an interrupt is generated for every small data unit.
Therefore, DMA is the most suitable method for transferring large MRI image blocks.
Recommended method:
MRI Scanner
Large image data
DMA Controller
Transfers data directly
Main Memory (RAM)
Temporary image buffer
Storage operation
SSD / PACS Storage
Permanent image storage
The CPU only needs to configure the transfer, manage the operation and respond to completion notifications.

9.6 Effect of Caching on Patient Record Look-up

Caching is the process of storing frequently accessed data in faster memory.
For instance, if a doctor repeatedly accesses the same patient's medical record, the database may retain frequently used pages in RAM.
Without caching, the system may need to retrieve data from SSD or network storage repeatedly.
With caching, frequently accessed information can be retrieved much faster.

Cache hit and cache miss

Cache hit: Requested data is already available in the cache.
Cache miss: Requested data is not available in the cache and must be retrieved from a slower storage level.

9.7 Alternative Concept: Interrupt-Driven I/O Instead of DMA

Suppose the MRI scanner transfers a 500 MB image using interrupt-driven I/O, generating one interrupt per word.

Comparison of DMA and interrupt-driven transfer

Feature	Interrupt per word	DMA
Image size	500 MB	500 MB
Transfer units	4-byte words	Large blocks
Number of notifications	131,072,000	Usually a small number of completion notifications
CPU involvement	Very high	Low
Memory buffer	Often small or repeated small buffers	Large block or ring buffers
Efficiency	Poor for large transfers	High

9.8 Conclusion

The memory hierarchy improves overall hospital-system performance by placing frequently accessed information in faster storage.

Patient records should be stored in fast and reliable databases, MRI images should use SSD and PACS storage, and monitoring data should be buffered in RAM before being stored permanently.

Interrupt-driven I/O is suitable for medical monitoring devices, whereas DMA is preferable for large MRI image transfers.

Caching further improves record retrieval performance by reducing repeated access to slower storage devices.

TASK 10 – MEASURE AND COMPARE

10.1 Introduction

Theoretical analysis explains how an operating system should behave, but practical measurement is required to understand its actual performance. Linux provides several tools for measuring system calls, CPU time and process creation performance.
The main tools used in this task are:

59. strace
60. time
61. fork()
62. pthread_create()

These tools help compare processes and threads and evaluate the efficiency of the operating system.
The measurements are useful for selecting a suitable process-management strategy for hospital applications.

10.2 Measuring System Calls Using strace

A system call is a request made by a user program to the operating system kernel.
Examples include:

63. read()
64. write()
65. openat()
66. close()
67. fork()
68. execve()
69. wait4()

The strace command can trace system calls made by a Linux process.
The -c option provides a summary containing:

70. Number of calls.
71. Total time spent in each system call.
72. Average time.

73. Errors.

Copy program in C

Create a file named copy.c.

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
   if (argc != 4)
   {
      printf("Usage: %s source destination buffer_size\n",
            argv[0]);
      return 1;
   }
   size_t size = (size_t)atoi(argv[3]);
   if (size == 0)
   {
      fprintf(stderr, "Invalid buffer size\n");
      return 1;
   }
```

These commands test buffer sizes of:

74. 1 KB.
75. 4 KB.
76. 64 KB.
77. 1 MB.

Expected observations

The number of read and write system calls generally decreases as the buffer size increases.

For instance, a 100 MB file copied using a 1 KB buffer requires approximately 102,400 read operations and a similar number of write operations at the application-buffer level.

Using a 1 MB buffer requires approximately 100 read operations and 100 write operations.

However, because the program uses fread() and fwrite(), the C library may perform internal buffering. Actual system-call counts will depend on the library implementation and buffering behaviour.
Illustrative comparison:

Buffer size	Approximate application read iterations	Approximate write iterations
1 KB	102,400	102,400
4 KB	25,600	25,600
64 KB	1,600	1,600
1 MB	100	100

10.3 Measuring User Mode and Kernel Mode Time

Linux provides the time command to measure program execution time.
Explanation:

Term Meaning

real Total elapsed wall-clock time

user CPU time spent executing user-space instructions

sys CPU time spent executing kernel code on behalf of the process

Difference between user and kernel time

User time: Time spent executing the application code.
Kernel time: Time spent executing operating-system services such as reading, writing, allocating memory and managing files.
For a file-copy program:

78. User time includes loop execution and buffer processing.
79. Kernel time includes file operations and system-call processing.
80. Real time includes CPU time and waiting for storage or other resources.

10.4 Measuring the Time to Create 1,000 Processes

A process has its own virtual address space and operating-system process-management structures.

To measure process creation, use fork().
Program: process_test.c

The program creates 1,000 child processes. Each child immediately terminates, and the parent collects the exit status of every child.
The measurement includes process creation, termination, scheduling and reaping

10.5 Measuring the Time to Create 1,000 Threads

Threads are smaller execution units that exist inside a process.
Threads share:

81. Program code.
82. Global variables.
83. Heap memory.
84. Open file descriptors.

Each thread has its own stack and execution state.

10.6 Comparison of Processes and Threads

Processes and threads differ in creation cost, memory requirements and isolation.

Feature	**Processes**	**Threads**
Creation	Usually more expensive	Usually less expensive
Memory	Separate virtual address spaces	Shared address space
Stack	Separate	Separate per thread
Heap	Separate	Shared
Communication	IPC required	Shared memory directly
Protection	Stronger isolation	Less isolation

Failure	Often isolated to one process	May affect entire process
Context switching	Often higher overhead	Often lower overhead

10.7 Illustrative Measurement Results

The following table demonstrates how experimental results can be presented.
Illustrative data only – replace with your own Linux measurements.

Experiment	Real time	User time	System time
1,000 processes	0.85 s	0.10 s	0.42 s
1,000 threads	0.18 s	0.04 s	0.08 s

10.8 Conclusion

Linux measurement tools provide useful evidence about operating-system performance.
strace -c identifies the system-call workload, while time measures real, user and kernel execution time.
The process and thread experiments help compare creation overhead and memory requirements.
For the hospital system, threads are useful for efficient internal parallelism, while separate processes are preferable for security-sensitive and independent services.

Task 10 – Measurement programs

A. Create and reap 1,000 processes

Save as task10_process_bench.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
```

```
int main(void)
{
    const int count = 1000;
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < count; i++) {
        pid_t p = fork();
        if (p < 0) { perror("fork"); return 1; }
        if (p == 0) _exit(0);
        int status;
        if (waitpid(p, &status, 0) < 0) {
            perror("waitpid"); return 1;
        }
    }

    clock_gettime(CLOCK_MONOTONIC, &end);
    double seconds = (end.tv_sec - start.tv_sec) +
                     (end.tv_nsec - start.tv_nsec) / 1e9;
    printf("%d fork/exit/wait cycles: %.6f seconds\n", count, seconds);
    return 0;
}
```

Compile/run: gcc -Wall -Wextra task10_process_bench.c -o procbench && ./procbench

B. Create and join 1,000 threads (POSIX pthreads)

Save as task10_thread_bench.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

#define COUNT 1000

static void *worker(void *arg)
{
    (void)arg;
    return NULL;
}
```

```
int main(void)
{
    pthread_t threads[COUNT];
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < COUNT; i++) {
        int rc = pthread_create(&threads[i], NULL, worker, NULL);
        if (rc != 0) {
            fprintf(stderr, "pthread_create failed at %d (error %d)\n", i, rc);
            return 1;
        }
    }

    for (int i = 0; i < COUNT; i++) {
        int rc = pthread_join(threads[i], NULL);
        if (rc != 0) {
            fprintf(stderr, "pthread_join failed (error %d)\n", rc);
            return 1;
        }
    }

    clock_gettime(CLOCK_MONOTONIC, &end);
    double seconds = (end.tv_sec - start.tv_sec) +
                     (end.tv_nsec - start.tv_nsec) / 1e9;
    printf("%d pthread create/join operations: %.6f seconds\n", COUNT,
seconds);
    return 0;
}
```

Compile/run: gcc -Wall -Wextra -pthread task10_thread_bench.c -o threadbench && ./threadbench

Measurement protocol

- Run each benchmark at least 5 times; report the median, not a fabricated value.
- Record OS/kernel version: uname -a; compiler version: gcc --version.
- Process timing: /usr/bin/time -v ./procbench
- Copy syscall comparison: strace -c ./task6_copy
- Use a table with run 1–5 and median. Results vary by CPU, Linux version, load, and compiler.

TASK 11 – RECOMMEND AND DEFEND THE OPERATING-SYSTEM PLAN

11.1 Introduction

A group of three hospitals requires a reliable and secure operating-system architecture to manage clinical operations.
The hospitals must support:

85. Patient registration.
86. Electronic health records.
87. MRI and CT imaging.
88. Laboratory information systems.
89. Pharmacy applications.
90. Patient monitoring.
91. Billing and administration.
92. Communication between departments.

Different hospital environments have distinct requirements.
For instance, a patient monitor requires predictable response time, whereas a billing application mainly requires accurate data processing and secure storage.
Therefore, using the same operating-system configuration for every environment would not be ideal.
A suitable plan should combine general-purpose operating systems, real-time systems, embedded operating systems and secure server infrastructure.

11.2 Recommended Operating-System Types

1. Patient Monitoring Environment

Patient-monitoring devices continuously measure important physiological parameters.
Examples:

93. Heart rate.
94. Blood pressure.
95. Oxygen saturation.
96. Respiratory rate.

Recommended OS: Embedded Real-Time Operating System (RTOS).
Examples include:

1. FreeRTOS.
2. Zephyr.
3. QNX.

Reason:

A monitoring system must respond to important events within predictable time limits.

An RTOS supports priority-based scheduling and bounded interrupt handling.

A normal desktop operating system may provide good average performance but cannot automatically guarantee hard real-time deadlines.

Required services:

1. Real-time task scheduling.
2. Interrupt handling.
3. Device drivers.
4. Memory management.
5. Communication interfaces.
6. Fault monitoring.

2. Hospital Database Servers

Database servers manage patient records, laboratory results, prescriptions and billing data.

Recommended OS: Linux Server.

Examples include Ubuntu Server, Debian or Red Hat Enterprise Linux.

Reasons:

1. Stable server environment.
2. Strong user and permission management.
3. Efficient memory management.
4. Support for database systems.
5. Networking and security facilities.
6. Monitoring and backup tools.

Required services:

1. Database management.
2. Authentication.
3. Authorization.
4. Logging.
5. Backup.
6. Network services.
7. Encryption.

3. MRI and Medical Imaging Environment

MRI and CT scanners require specialized hardware interfaces and reliable data transfer.

Recommended OS: Vendor-supported embedded or real-time operating system for scanner control, with a Linux-based or vendor-supported PACS server for image storage and retrieval.

Reasons:

1. Predictable device-control behaviour.
2. Reliable data transfer.
3. Hardware-specific driver support.
4. High-speed image processing.
5. Integration with medical imaging systems.

DMA should be used where supported for large image transfers.

11.4 Operating-System Services, Protection and I/O Methods

The OS must provide the correct services for each environment.

Environment	OS type	Main services	Protection	I/O method
Patient monitors	RTOS	Scheduling, interrupts	Device isolation	Interrupt-driven
Database servers	Linux Server	Files, networking, memory	Access control, encryption	Interrupts and DMA where applicable
MRI scanners	Embedded/vendor OS	Device control	Hardware and process protection	DMA
PACS servers	Linux/vendor-supported OS	Image storage, networking	Authentication and authorization	DMA and storage I/O
Desktop systems	Managed desktop OS	GUI, applications	User permissions, endpoint security	Interrupt-driven
Network	Hardened	Routing,	Network	Device

devices	appliance OS	firewalling	access controls	interrupts/DMA

Security mechanisms

The following mechanisms should be implemented:

1. Role-Based Access Control (RBAC).
2. Multi-factor authentication for privileged access.
3. Encryption of data at rest and in transit.
4. Network segmentation.
5. Regular security updates.
6. Centralized audit logging.
7. Secure backup and disaster recovery.
8. Least-privilege permissions.

For instance, nurses may access patient-monitoring records, but only authorized clinicians should modify certain clinical decisions.
Similarly, administrative staff should not have unrestricted access to confidential clinical data.

11.5 Reliability and Performance Considerations

A hospital operating-system plan must prioritize availability and response time.

Real-time response

Monitoring systems must prioritize abnormal readings and emergency alerts.
The RTOS should use appropriate priority scheduling and bounded interrupt processing.

Database reliability

The database should support:

1. Transaction processing.
2. Replication.
3. Backup.
4. Recovery.
5. Consistency.
6. Access auditing.

Network reliability

The three hospitals should have redundant network connections where practical.

This reduces the risk of losing communication when one connection fails.

Storage reliability

Patient records and medical images should be backed up regularly.

A combination of SSD storage, replicated storage and offline or immutable backups can protect against hardware failure and ransomware.

11.6 Will the Plan Support Three Hospitals?

Yes. The proposed architecture is suitable for a group of three hospitals if the systems are designed with scalability and interoperability in mind.

Each hospital should have:

1. Local clinical application services.
2. Local monitoring infrastructure.
3. Secure access to patient records.
4. Local backup and recovery capabilities.
5. A connection to the shared inter-hospital network.

Centralized services can be used for common functions, but essential local operations should not completely depend on a remote server.

For instance, if the network connection between Hospital A and Hospital B fails, Hospital A should still be able to access its local patient-monitoring systems.

The design should also support standards-based information exchange, such as DICOM for medical images and suitable healthcare interoperability standards such as HL7 FHIR for clinical records

11.7 Conclusion

The proposed hybrid operating-system plan provides a balance between performance, reliability, security and scalability.

Real-time systems provide predictable monitoring responses, while Linux servers provide stable database and application services.
Virtualization can reduce hardware costs, but critical medical systems should not be exposed to unnecessary scheduling delays or shared-resource failures.
The architecture can support all three hospitals if local services, secure communication, redundancy and disaster recovery are properly implemented.

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN

12.A IMPACT ANALYSIS –

12.A.1 Introduction

The original hospital design mainly focused on systems located inside the hospital.

The new requirements introduce external devices and cloud-based patient records.

This creates additional requirements for:

1. Internet communication.
2. Remote device management.
3. Authentication.
4. Encryption.
5. Distributed data management.
6. Cloud availability.
7. Network reliability.
8. Remote security monitoring.

The following operating-system types, services and protection mechanisms are affected.

1. Embedded Operating Systems for Home Monitoring Devices

Home monitoring devices collect patient information outside the hospital.

Examples include:

1. Wearable heart-rate monitors.
2. Blood pressure monitors.
3. Pulse oximeters.
4. Connected glucose monitors.

These devices require an embedded operating system that can manage sensors, networking and power effectively.

An RTOS such as FreeRTOS or Zephyr may be appropriate, depending on the device's certification and safety requirements.

The device should support:

1. Sensor management.
2. Data collection.
3. Local buffering.
4. Network communication.
5. Secure boot.

6. Firmware updates.
7. Error detection.

The system should continue collecting readings even when internet connectivity is temporarily unavailable.

2. Distributed Operating-System Services

The original design mainly used local hospital servers.

Now patient records must be shared through a cloud service.

This requires distributed-system services such as:

1. Remote authentication.
2. Distributed database access.
3. Data replication.
4. Synchronization.
5. Load balancing.
6. API communication.
7. Fault tolerance.
8. Backup and recovery.

A distributed architecture allows authorized users at different hospitals to access shared records.

Impact analysis table

Original component	**Changed requirement**	**Required modification**
Local patient monitors	Home monitoring devices	Embedded RTOS and secure networking
Local database	Cloud-shared records	Distributed cloud database
Local network	Internet communication	Secure gateways and encrypted connections
User authentication	Remote users and devices	Strong identity management
Local backup	Cloud-based shared data	Replication and disaster recovery
Hospital security	External attack exposure	Zero-trust principles and monitoring
Local storage	Shared cloud records	Data synchronization and consistency

Evaluation: The original architecture provides a good foundation, but it does not fully support remote devices and cloud-shared patient records without modification.

12.B EVALUATION OF THE ORIGINAL DESIGN – 2 MARKS

Can the Original Design Support Home Monitoring Devices?

The original design uses embedded RTOS platforms for patient monitoring.

This is a good starting point because home monitoring devices also require efficient sensor management and predictable processing.

However, the original design was mainly intended for devices connected to hospital infrastructure.

Home devices introduce additional challenges.

Response time

Home monitoring readings must be processed locally before being transmitted.

For instance, if a patient's oxygen saturation becomes dangerously low, the device should detect the abnormal reading locally rather than depending entirely on a cloud connection.

The local device should:

1. Read the sensor.
2. Check the measurement.
3. Detect abnormal values.
4. Generate a local alert where appropriate.
5. Transmit the reading to the cloud or hospital.
6. Retry communication if the network is unavailable.

This reduces dependence on internet response time.

Internet latency can vary due to network congestion, distance and service-provider problems.

Therefore, a cloud server should not be the only location responsible for detecting urgent medical events.

Reliability

Home internet connections may be unstable.

If the connection is lost, the device should temporarily store readings in local memory.

After reconnection, it should transmit the buffered data.
This process is known as store-and-forward communication.
It improves reliability because temporary network failure does not automatically cause permanent data loss.

Security

Home devices communicate over public networks.
This increases the risk of:

1. Unauthorized access.
2. Data interception.
3. Device impersonation.
4. Malware.
5. Firmware manipulation.
6. Denial-of-service attacks.

Therefore, the original design needs stronger remote-device authentication and encrypted communication.

12.C ADAPTATION OR REDESIGN

Revised Operating-System Plan

The revised design will use a hybrid distributed architecture.
It combines:

1. Embedded RTOS for home monitoring devices.
2. Linux or suitable embedded operating systems for hospital gateways.
3. Cloud infrastructure for shared patient records.
4. Linux-based application servers.
5. Secure distributed databases.
6. Local buffering and emergency alert processing.

Cloud-Based Patient Record Design

The patient record system will be moved to a secure cloud platform.
The cloud service will provide:

1. Centralized patient record storage.

2. Database replication.
3. Backup and recovery.
4. Authentication.
5. Role-based authorization.
6. Secure APIs.
7. Audit logging.
8. Monitoring and scaling.

Partner hospitals can access the same authorized patient records.
This reduces the need for manual transfer of patient information between hospitals.

Data Sharing and Interoperability

The system should use healthcare interoperability standards.
Examples:
HL7 FHIR: Supports structured exchange of healthcare information.
DICOM: Supports storage and exchange of medical images.
These standards improve compatibility between different hospital applications and medical systems.

12.D EVIDENCE-BASED DEFENCE

12.D.1 Why the Revised Design Is Better

The revised design is recommended because it supports both the original hospital requirements and the new requirements for remote monitoring and cloud-based patient records.
It improves scalability, accessibility and data sharing while maintaining local processing for critical monitoring operations.
The decision is based on the following technical considerations.

1. Response Time

Home monitoring devices should process abnormal readings locally rather than waiting for a cloud response.
For instance, when a device detects an abnormal reading, it should immediately activate the appropriate local alert mechanism.
The reading can then be transmitted to the cloud for storage and review by healthcare professionals.

This approach reduces the effect of internet latency on emergency detection.

2. Reliability

The revised design uses local buffering and cloud replication.
If a home internet connection fails, readings are temporarily stored on the device.
When connectivity returns, the buffered data is transmitted.
Cloud replication and backups also reduce the risk of losing patient records due to a single server failure.
However, replication must be combined with independent backups because replication alone can also reproduce accidental deletions or corrupted data.

3. Security

The revised design introduces stronger security controls.
All remote devices must authenticate before sending data.
Patient records must be encrypted during transmission and storage.
Access permissions must be based on user roles.
This reduces the possibility of patient-data leakage.

4. Scalability

The cloud system can support increasing numbers of patients and monitoring devices.
If the hospital adds more home-monitoring devices, cloud resources can be increased according to demand.
This is more flexible than purchasing additional physical servers for every increase in workload.

5. Cost and Maintenance

Cloud services reduce the need for the hospital to purchase and maintain all physical infrastructure.
However, cloud services introduce recurring subscription, networking and data-transfer costs.
Therefore, the hospital should evaluate the total cost of ownership instead of assuming cloud computing is always cheaper.

12.D.2 Comparison of Original and Revised Designs

Evaluation factor	Original design	Revised design
Monitoring	Mainly hospital-based	Hospital and home-based
Operating system	Embedded RTOS and Linux	RTOS, Linux and cloud infrastructure
Patient records	Local/inter-hospital servers	Shared cloud database
Communication	Mainly hospital network	Internet and secure APIs
Response time	Mostly local	Local processing plus cloud access
Reliability	Local backup and redundancy	Local buffering, cloud replication and backup
Security	Internal access control	End-to-end protection and remote authentication
Scalability	Limited by local hardware	Cloud resources can scale
Data sharing	Between three hospitals	Shared cloud access for partner hospitals

12.D.3 Final Decision

Recommended: MODIFY the original design
I recommend modifying the original hospital architecture rather than completely redesigning it.
The original design already uses embedded real-time operating systems for monitoring devices and Linux servers for patient records and applications.
These are suitable foundations for the new requirements.
The following modifications are necessary:

1. Introduce embedded RTOS-based home monitoring devices.
2. Add secure internet communication using TLS.
3. Introduce cloud-based patient record storage.
4. Implement distributed database services and replication.
5. Use strong authentication and role-based access control.
6. Maintain local buffering and emergency alert processing.
7. Provide backup and disaster recovery.
8. Use HL7 FHIR and DICOM for healthcare data exchange where appropriate.
9. Implement centralized security monitoring.
10. Maintain local services for essential hospital operations during cloud or internet failures.

This solution provides a balance between performance, reliability, security, scalability and cost.

12.D.4 Overall Conclusion

The introduction of home monitoring devices and cloud-based patient records significantly changes the requirements of the hospital operating-system architecture.

The original design is suitable for local hospital operations but requires additional services to support remote monitoring and distributed data access.

An embedded RTOS should be used in home monitoring devices to provide predictable sensor processing and local alert generation.

Linux-based hospital servers and cloud infrastructure should manage patient records, applications, communication and shared information.

Cloud databases improve accessibility and scalability, while local buffering and replication improve reliability.

Strong encryption, authentication, authorization and audit logging are essential because medical information is highly confidential.

Therefore, modifying the original hybrid architecture is the most appropriate decision. It preserves the strengths of the original design while introducing the additional capabilities needed for remote healthcare and cooperation between partner hospitals.

Task 12 – Possible supporting artifacts (design-dependent)

The PDF asks you to re-evaluate the hospital architecture for remote monitoring and cloud records. It does not provide a single prescribed C program. Depending on your chosen prototype, include artifacts such as:

- Dockerfile for a service prototype
- YAML deployment/configuration file
- Shell script to launch/test services
- A small client/server or data-transfer program, if your report claims you implemented one

Do not claim remote monitoring/cloud execution was tested unless you actually run it and preserve output/screenshots.

GitHub folder suggestion

Task05/task5_syscall_demo.c
Task06/task6_copy.c
Task07/task7_process.c
Task10/task10_process_bench.c
Task10/task10_thread_bench.c
Task12/ (only the actual configuration/scripts used)
README.md (gcc commands, Linux version, steps, actual results)